Software Architecture and Design Document

Web-Based ML Model Evaluator (WBMME)

Complete document: Architecture (Section 3) and Design (Section 4)

Project	Web-Based ML Model Evaluator
Course	UE24CS341A Software Engineering, B.Tech CSE, V Semester
Repository	SE_498_515_534_911
Document	Software Architecture and Design Document (SAD), complete
Version / Status	1.0 / Integrated, for team review
Author: Sections 1, 2, 3.1, 3.2, 4 and 5	Member 4
Author: Sections 3.3 to 3.9	Member 3
Date	13-10-2026
Related documents	SRS (Members 1 and 2), Test Plan (Members 1 to 4), RTM

Revision History

Version	Date	Author	Change summary	Approval
0.1	12-10-2026	Member 4	Initial draft of Part B: cover, Sections 1, 2, 3.1, 3.2, 4.1 to 4.6 and Appendices	Pending team review
0.2	12-10-2026	Member 4	Aligned with SRS Part B: four-model catalogue, upload limits, job quotas and timeout, login and session rules, macro-averaged metrics; added MLE-NF-006, NF-007, SR-006, SR-007 and MLE-SO-001 to 003	Pending team review
1.0	13-10-2026	Members 3 and 4	Merged Part A (Sections 3.3 to 3.9, Member 3) with Part B (Member 4) into one document; figures renumbered 1 to 7; cover, contents, references and open issues updated	Pending team review

Approvals

Role	Name	Signature / Date
Course Coordinator		
Faculty guide		
Team representative		

Contents

1. Introduction

2. Document Overview

3. Architecture (3.1 Goals and Constraints, 3.2 Stakeholders, 3.3 Component Diagram, 3.4 Component Descriptions, 3.5 Pattern, 3.6 Technology Stack, 3.7 Risks, 3.8 Traceability, 3.9 Security Architecture)

4. Design (4.1 Design Overview, 4.2 Sequence Diagrams, 4.3 API Design, 4.4 Error Handling, Logging and Monitoring, 4.5 UX Design, 4.6 Open Issues)

5. Appendices (5.1 Glossary, 5.2 References, 5.3 Tools, A: Design reference index, B: Repository layout)

1. Introduction

1.1 Purpose

This Software Architecture and Design Document (SAD) describes how the Web-Based ML Model Evaluator (WBMME) is structured and how its parts are designed. It turns the requirements in the SRS into modules, data structures, interactions and interfaces that the team can implement, the testers can verify and the evaluators can review.

It covers the two design deliverables required by the Jackfruit guidelines: the high-level architecture document and the software design document (UML, API definition, modular design and division of work within the team).

1.2 Scope

This document covers WBMME v1.0: user registration, login and role-based access; CSV dataset upload, profiling and preparation; model selection, configuration and training; evaluation metrics and charts; experiment history, comparison and report export; administration; and logging.

It does not cover deep-learning models, uploading of pre-trained model files, real-time prediction serving, mobile layouts or production-scale deployment. As in the SRS, the application is intended for educational and experimental evaluation of models, not for certifying production suitability.

1.3 Definitions and acronyms

Terms defined in SRS Section 1.4 are used with the same meaning. Additional terms used in this document:

Term	Meaning
Run	One recorded execution of a chosen model, parameters, preprocessing and split on a dataset, with its status and metrics.
Artifact	A trained model file written by the system (joblib) and stored in the owner's private directory.
Pipeline	The ordered stages Upload, Validate, Profile, Prepare, Split, Train, Evaluate and Record.
Service	A Python class holding business logic with no dependency on HTTP; called by controllers.
Controller	A Django view that validates a request, checks authorisation, calls services and returns JSON.
Model registry	The catalogue that maps a model name to its scikit-learn class and allowed hyperparameters.
MVT	Model-View-Template, the Django variant of the MVC pattern.
RBAC	Role-based access control.
CSRF	Cross-Site Request Forgery.
DS-ID	Design reference ID (for example DS-Data-01) used in SAD Section 3.8 and Appendix A to trace a requirement to its design.

Term	Meaning
MLE-F / MLE-NF / MLE-SR	Functional, non-functional and security requirement IDs defined in SRS Part B.

1.4 References

- WBMME Software Requirements Specification (SRS), v0.1, Members 1 and 2.
- WBMME Software Test Plan, v0.1, Members 1 to 4.
- Jackfruit SE Mini-Project Guidelines 2026, PES University (UE24CS341A).
- IEEE Std 1016-2009, Systems Design: Software Design Descriptions.
- OWASP Top 10 and OWASP Application Security Verification Standard.
- Django 5 and scikit-learn documentation (framework and library behaviour).

2. Document Overview

2.1 Organisation

Section	Content	Owner	Status
Cover, 1, 2	Front matter, purpose, scope, definitions, how to read the document	Member 4	Included
3.1, 3.2	Architecture goals and constraints; stakeholders	Member 4	Included
3.3 to 3.9	Component diagram, component descriptions, pattern, technology stack, risks, traceability, security architecture	Member 3	Included
4.1	Design overview: modules, pipeline, data model, key decisions	Member 4	Included
4.2	Sequence diagrams: login, upload, train and view metrics	Member 4	Included
4.3	API design: conventions, endpoint list, detailed contracts	Member 4	Included
4.4	Error handling, logging and monitoring	Member 4	Included
4.5	UX design: screen map, inventory, wireframes	Member 4	Included
4.6, 5	Open issues and next steps; appendices (glossary, references, tools)	Member 4	Included

2.2 Related Documents

Software Requirements Specification (SRS), Software Test Plan (STP) and the Requirements Traceability Matrix (RTM). See Section 2.5 for how they relate to this document.

2.3 How to read this document

Reader	Start with
Developer	Section 3.4 (components), then 4.1 (modules and pipeline), 4.3 (API contracts) and 4.4 (errors and logging).
Tester	Appendix A (requirement to design to test traceability), 4.3 (expected responses) and 4.4 (error catalogue).
Reviewer / faculty	Section 3.1 (goals and constraints), 3.5 (pattern rationale), 4.2 (sequences) and 4.6 (open issues).
Designer / UX	Section 4.5 and the screens listed there.

2.4 Conventions

- **Requirement IDs.** MLE-F-nnn (functional), MLE-NF-nnn (non-functional) and MLE-SR-nnn (security) are final and match SRS Part B (Member 2), SAD Section 3.8 and the Test Plan. Appendix

A lists every ID, including the new MLE-NF-006, NF-007, SR-006, SR-007 and objectives MLE-SO-001 to 003.

- **Design reference IDs.** DS-Area-nn, as defined in SAD Section 3.8. Appendix A lists where each is designed in this document.
- **Figures.** Figures are numbered 1 to 7 in order of appearance: Figure 1 is in Section 3.3 and Figures 2 to 7 are in Section 4.
- **Proposed values.** Limits, quotas and defaults marked as proposed are design choices awaiting team confirmation (see Section 4.6).
- **Notation.** Interfaces are JSON over HTTP. Times are ISO 8601 in UTC. Diagrams follow UML sequence and entity-relationship notation.

2.5 Relationship to other documents

The SRS states what the system must do, this SAD states how it is built, and the Test Plan states how each requirement is verified. The Requirements Traceability Matrix (RTM) links the three through requirement IDs, design references and test case IDs. Appendix A extends the RTM with the design column.

3. Architecture

3.1 Goals and Constraints

3.1.1 Architecture goals

ID	Goal	How the architecture meets it	Requirements
G1	Correct, reproducible evaluation	Preprocessing is fitted on the training split only; every run stores its dataset, model, parameters, preprocessing, split and random seed; train and test metrics are both reported.	MLE-F-006 to F-013
G2	Confidentiality and integrity of user data	Ownership check on every query and file path; validated uploads; only system-created artifacts are loaded; audit log of sensitive actions.	MLE-SR-001 to SR-005
G3	Responsive under load	Training runs as a background job with status polling; upload and row limits; stateless API; paginated lists.	MLE-NF-001 to NF-003
G4	Modular and extensible	Layered design with service classes behind controllers; a model registry so a new algorithm is added by one registry entry; pipeline stages with a common interface.	Guidelines
G5	Testable	Services have no HTTP dependency, so unit tests need no browser; layers can be stubbed; coverage target 70% or higher for service modules.	Test Plan 5, 14
G6	Easy for a first-time user	Guided linear workflow, safe defaults, plain-language errors, visible progress.	MLE-NF-005
G7	Portable	Docker image with pinned dependencies; no OS-specific code.	MLE-NF-004
G8	Deliverable in two sprints	Single deployable unit; minimal end-to-end flow first; explainability and PDF export deferrable.	Test Plan 7

3.1.2 Constraints

ID	Constraint	Source	Design impact
C1	Agile process only; backlog and user stories in Jira or GitHub	Guidelines	Modules map to the backlog epics (WBMME-4 to WBMME-11); work is delivered in two sprints.
C2	CI/CD, code review and security validation are mandatory	Guidelines	GitHub Actions runs tests and linting on every pull request; pull-request reviews before merge.
C3	Every member implements a full functionality; commits are checked	Guidelines	Modules in 4.1.2 are separable so each member can own one end to end.
C4	Python 3.11, Django 5, pandas, scikit-learn, joblib, SQLite or PostgreSQL	SRS 2.4, SAD 3.6	Technology choices are fixed; no other backend language.

ID	Constraint	Source	Design impact
C5	The browser never touches the database or file store directly	SRS 2.5	All access goes through the API layer and services.
C6	CSV only; four catalogue models; 10 MB, 20 to 50,000 rows, 2 to 100 columns, 20 datasets per user	SRS Tables 4.2-1 and 4.3-1	Validation rules in 4.3.3; model registry in 4.3.4.
C7	Datasets and artifacts are stored outside the web root, per user	SRS 2.4, 2.5	File store layout in 4.1.4 and Appendix B.
C8	Two sprints, finishing by 30-Oct-2026	Test Plan 7	Optional features (SHAP, Celery, PDF export) are deferrable.
C9	Desktop browsers only	SRS 2.4	No responsive mobile layouts are designed.
C10	Single deployable unit, no microservices	SAD 3.5	Services are Python modules in one Django project.

3.1.3 Main trade-offs

- **Threads versus Celery for training.** Threads need no extra infrastructure and suit the team size, but jobs are lost on restart. The design persists run status in the database and marks interrupted runs FAILED at start-up. Celery remains an upgrade path behind the same `JobRunner` interface.
- **Polling versus WebSockets.** Polling `/api/evaluate/{run_id}` every 2 seconds is simpler to build and test; the delay is acceptable for jobs of up to a minute.
- **Preprocessing inside the training request.** Missing-value, encoding and split options travel with `/api/train` rather than being saved as a separate prepared dataset. This avoids data leakage and keeps one run reproducible from one record.

3.2 Stakeholders

Stakeholder	Description	Main concerns	Addressed by
Data Scientist	Experienced ML user who uploads datasets, trains and compares models.	Choice of algorithms and parameters, trustworthy metrics, run comparison, export.	4.1.3 pipeline, 4.3.4 model catalogue, 4.3.6 runs and compare.
Student	Learner using the tool for coursework and experiments.	Clear guidance, understandable errors, safe defaults.	4.5 UX design, 4.4 error messages.
Administrator	Operates the platform, manages users and datasets, reviews logs.	Access control, audit trail, system health, no exposure of user data.	3.9, 4.3.7 admin API, 4.4 logging and monitoring.
Development team (Members 1 to 4)	Builds, tests and documents the system.	Clear module boundaries, stable APIs, parallel work without merge conflicts.	4.1.2 modules, 4.3 API contracts, Appendix B.
Repository owner / reviewer	Reviews pull requests and protects the main branch.	Quality gates, traceability from requirement to code to test.	Constraint C2, Appendix A.

Stakeholder	Description	Main concerns	Addressed by
Course faculty and evaluators	Assess documents, code understanding and the demo.	Complete SDLC artefacts, justified decisions, working demo.	Sections 2, 3.1, 4.2 and 4.6.

3.3 Component (UML) Diagram

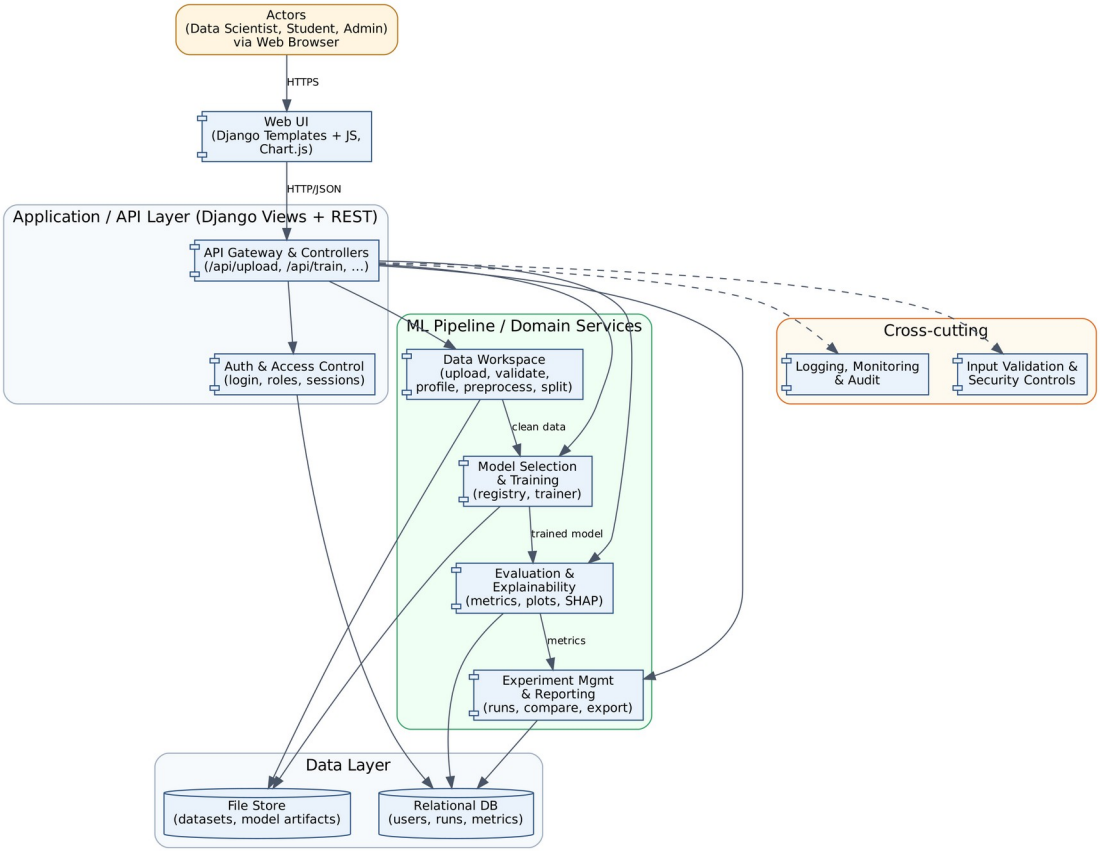


Figure 1: Component diagram. Solid arrows are runtime dependencies; dashed arrows are cross-cutting concerns applied by the API layer.

3.4 Component Descriptions

Component	Responsibility	Maps to epic
Web UI	Renders pages for upload, model selection, training status, results and history. Charts (confusion matrix, metric bars) use Chart.js. Contains no business logic.	WBMME-4 to WBMME-7 (UI parts)
API Gateway & Controllers	Single entry point for all requests (/api/upload, /api/train, /api/evaluate, /api/runs). Validates input, checks authorisation, delegates to services, returns JSON.	All

Component	Responsibility	Maps to epic
Auth & Access Control	Registration, login, session handling and role-based access (Data Scientist, Student, Admin). Ensures users only see their own datasets and runs.	WBMME-11
Data Workspace	Accepts CSV upload, validates type, size and schema, profiles the data, handles missing values and categorical encoding, and creates the train/test split.	WBMME-4
Model Selection & Training	Holds a registry of supported scikit-learn models and their hyperparameters. Trains the selected model(s) on the prepared data and stores the trained artifact.	WBMME-5
Evaluation & Explainability	Computes accuracy, precision, recall, F1, ROC-AUC and the confusion matrix. Lists feature importance for models that provide it (Logistic Regression, Decision Tree, Random Forest).	WBMME-6
Experiment Management & Reporting	Records every run (dataset, model, parameters, metrics), supports side-by-side comparison, and exports a run or comparison as a CSV file.	WBMME-7
Logging, Monitoring & Audit	Structured logs of requests, training jobs, failures and admin actions. Never logs raw data values or credentials.	WBMME-11 (and WBMME-9 for deployment logs)
Input Validation & Security Controls	Central file validation, request-size limits, rate limiting, CSRF protection and safe model serialisation.	WBMME-11
Relational DB	Stores users, roles, dataset metadata, runs and metrics.	WBMME-4 to WBMME-7
File Store	Stores uploaded datasets and trained model artifacts in per-user directories outside the web root.	WBMME-4, WBMME-5

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: Layered architecture (Presentation, Application/API, ML Pipeline/Domain, Data) with the Django MVT structure inside the web tier and a pipe-and-filter style ML pipeline (Upload, Preprocess, Train, Evaluate, Record).

Rationale:

- Separation of concerns: UI changes do not affect ML code, and a model can be swapped without touching the views.
- Matches the workflow. The ML flow is naturally sequential, so each stage is a replaceable component with a defined input and output.
- Fits the team: four students and a two-sprint schedule. A single deployable unit keeps CI/CD, debugging and the demo simple.
- Testable: each layer can be unit-tested with stubs, which supports the Test Plan's unit, integration and system levels.

Alternatives considered and rejected:

Alternative	Reason rejected
Microservices	Needs service discovery, inter-service auth and multiple deployments. Too much overhead for the team size and the sprint length.
Single-file monolith (no layers)	Fastest to start, but couples UI and ML code and makes parallel work on four branches cause merge conflicts.
Client-side ML (in-browser)	Limits model choice and dataset size, and is hard to secure and test.

3.6 Technology Stack and Data Stores

Area	Choice	Why
Language / Backend	Python 3.11, Django 5	Same language as the ML libraries; built-in auth, ORM, CSRF protection.
API style	Django views returning JSON (Django REST Framework optional)	Simple, documented endpoints for upload, train, evaluate.
Frontend	Django templates, JavaScript, Chart.js	No separate build chain; enough for dashboards.
ML libraries	pandas, scikit-learn, joblib	Standard, well documented, many classic models.
Relational DB	SQLite (development), PostgreSQL (optional deployment)	Zero-setup locally; production-ready option.
File store	Local per-user directories (outside web root)	Holds datasets and model artifacts; can later move to object storage.
Background jobs	Python background worker (Celery is optional and not committed in the SRS)	Keeps the UI responsive during long training.
CI/CD & quality	GitHub Actions, pytest, pytest-cov, flake8/SonarQube	Required by the guidelines; runs tests on every PR.
Packaging / Deployment	Docker (WBMME-9)	Portable, reproducible environment.
Security libs	Django auth, bcrypt/PBKDF2, HTTPS (TLS 1.2+)	Standard, audited components.

3.7 Risks and Mitigations

ID	Risk	Severity	Mitigation
R1	Long training times on large datasets block the UI or time out	High	Limit dataset size and rows; run training in a background job; show progress; set a timeout.
R2	Malicious or malformed uploads (wrong type, huge file, formula injection in CSV)	High	Whitelist .csv, check MIME type and size, parse with pandas in a safe mode, never execute content.

ID	Risk	Severity	Mitigation
R3	Unsafe loading of pickled/joblib model files can run arbitrary code	High	Only load artifacts the system itself created; store them per user; never accept uploaded model files.
R4	Data leakage or overfitting gives misleading results	Medium	Fit preprocessing only on the training split; report train and test metrics; document the method.
R5	Team members unfamiliar with Git/Django cause delays and merge conflicts	Medium	Branch per feature, small PRs, reviews by repo owner, pair on first PR, shared task board.
R6	Different library versions break the build between machines	Medium	Pin versions in requirements.txt; use Docker and CI to catch drift.
R7	Users' datasets visible to other users	High	Per-user ownership checks on every query and file path; tests for access control.
R8	Sprint timeline is too short for all epics	High	Prioritise by epic order; implement a minimal end-to-end flow first; defer feature importance and CSV export if needed.

3.8 Traceability to Requirements

Req ID	Requirement (short)	Architecture component	Design reference
MLE-F-001	User registration and login	Auth & Access Control	DS-Auth-01 (4.2 login sequence)
MLE-F-002	Role-based access (Data Scientist, Student, Admin)	Auth & Access Control	DS-Auth-02
MLE-F-003	Upload CSV dataset	Data Workspace, File Store	DS-Data-01 (4.2 upload sequence, /api/upload)
MLE-F-004	Preview and profile the dataset	Data Workspace	DS-Data-02
MLE-F-005	Handle missing values and encode categories	Data Workspace	DS-Data-03
MLE-F-006	Configure train/test split	Data Workspace	DS-Data-04
MLE-F-007	Select a model from the catalogue	Model Selection & Training	DS-Model-01
MLE-F-008	Configure hyperparameters	Model Selection & Training	DS-Model-02
MLE-F-009	Train a model and show status	Model Selection & Training	DS-Model-03 (/api/train)
MLE-F-010	Compute accuracy, precision, recall, F1	Evaluation & Explainability	DS-Eval-01 (/api/evaluate)
MLE-F-011	Show confusion matrix and plots	Evaluation & Explainability, Web UI	DS-Eval-02
MLE-F-012	Feature importance / explanation	Evaluation & Explainability	DS-Eval-03
MLE-F-013	Save each run (history)	Experiment Mgmt & Reporting, DB	DS-Exp-01

Req ID	Requirement (short)	Architecture component	Design reference
MLE-F-014	Compare runs side by side	Experiment Mgmt & Reporting	DS-Exp-02
MLE-F-015	Export results report	Experiment Mgmt & Reporting	DS-Exp-03
MLE-F-016	Admin manages users and datasets	Auth & Access Control, Web UI	DS-Admin-01
MLE-NF-001	Upload of up to 10 MB processed in 5 s or less	Data Workspace	Performance budget in 3.7 R1
MLE-NF-002	Training of standard models on up to 50,000 rows completes in 60 s or less	Model Selection & Training	Background job design
MLE-NF-003	Pages load in 3 s or less for 20 concurrent users	API Gateway, Web UI	Layered stateless API
MLE-NF-004	Runs on Windows, Linux and macOS via Docker	Packaging	3.6 Docker
MLE-NF-005	Usable by a first-time user without training	Web UI	4.5 UX Design
MLE-NF-006	Failed jobs are contained; saved data survives restart	Model Selection & Training, Relational DB	Background job design (SAD 3.6, risk R1)
MLE-NF-007	Page-load limits with 20 datasets and 100 runs per user	Experiment Management & Reporting, Relational DB	Scalability / pagination design
MLE-SR-001	Passwords stored hashed	Auth & Access Control	SAD 3.9 Spoofing
MLE-SR-002	All traffic over HTTPS (TLS 1.2+)	Deployment / Gateway	SAD 3.9 Information disclosure
MLE-SR-003	Uploads validated for type, size and content	Input Validation & Security Controls	SAD 3.9 Tampering
MLE-SR-004	Only system-created model artifacts are loaded	Model Selection & Training	SAD 3.9 Tampering / EoP
MLE-SR-005	Per-user data isolation and audit log of actions	Auth, Logging & Audit	SAD 3.9 Repudiation, Info disclosure
MLE-SR-006	Request, job and run-time limits against abuse	Input Validation & Security Controls, API Gateway, Model Selection & Training	SAD 3.9: Denial of service
MLE-SR-007	Session expiry, login throttling and CSRF protection	Auth & Access Control, Input Validation & Security Controls	SAD 3.9: Spoofing
MLE-SO-001	Confidentiality	Auth & Access Control, File Store, API	SAD 3.9: Information disclosure
MLE-SO-002	Integrity	Input Validation & Security Controls, Model Selection & Training	SAD 3.9: Tampering

Req ID	Requirement (short)	Architecture component	Design reference
MLE-SO-003	Availability	Input Validation & Security Controls, API Gateway, Model Selection & Training	SAD 3.9: Denial of service

3.9 Security Architecture

Security objectives (to match SRS section 5.1.1): (1) Confidentiality of each user's datasets, models and results. (2) Integrity of uploaded data, trained models and reported metrics. (3) Availability of the service against abusive or oversized requests. These are delivered under the Core Platform & Security epic (WBMME-11).

Threat modelling with STRIDE:

Threat	ML Evaluator scenario	Mitigation (component)
Spoofing	Attacker logs in as another user or reuses a stolen session	Hashed passwords (PBKDF2/bcrypt), session expiry, login throttling, secure and HttpOnly cookies (Auth)
Tampering	Malicious CSV or modified model file changes results or runs code	File type, size and content validation; store artifacts outside web root; load only system-generated artifacts; no unpickling of user files (Input Validation, Model Training)
Repudiation	User denies deleting a dataset or an admin denies a change	Audit log of logins, uploads, trainings, deletions and admin actions with user ID and timestamp (Logging & Audit)
Information disclosure	One user reads another user's datasets or runs; sensitive data in logs or errors	Ownership check on every query and file path; HTTPS (TLS 1.2+); generic error messages; no raw data in logs (API, Auth, Logging)
Denial of service	Huge uploads or many concurrent training jobs exhaust the server	Upload size limit, row/column limits, rate limiting, per-user job quota, training timeout (API, Input Validation)
Elevation of privilege	Student reaches admin pages or APIs	Role-based access control on every view and endpoint; admin routes separated; deny by default (Auth)

Trust boundaries: (a) browser to server (untrusted input, HTTPS required); (b) API layer to ML services (all data already validated); (c) application to file store and database (access only through the data layer).

4. Design

4.1 Design Overview

4.1.1 Design approach

WBMME follows the layered architecture chosen in SAD Section 3.5: a Presentation layer (Django templates and JavaScript), an Application/API layer (controllers), an ML pipeline and domain layer (services) and a Data layer (database and file store). The design rules below keep the layers apart.

- **Thin controllers.** A controller validates input, checks the session and role, calls one service method and shapes the JSON response. It contains no ML or file logic.
- **Services own the logic.** Services are plain Python classes with explicit inputs and outputs, so they can be unit-tested without HTTP.
- **Owner-scoped data access.** Every query goes through a manager that filters by the requesting user's ID. Admins use a separate, narrower interface.
- **Model registry (strategy pattern).** The registry maps a model name to a scikit-learn class, its default parameters and the allowed range of each hyperparameter. Training code never hard-codes an algorithm.
- **Pipeline stages.** Each stage takes a context object and returns an updated one, so stages can be tested alone or replaced.

4.1.2 Module structure

Module	Responsibility	Key classes and functions	Component (3.4)
accounts	Users, roles, registration, login, logout, login throttling	`User` (role field), `AuthService`, `role_required` decorator, `PasswordPolicy`	Auth and Access Control
datasets	Upload validation, storage, profiling, missing values, encoding, split	`Dataset`, `UploadValidator`, `DatasetProfiler`, `Preprocessor`, `Splitter`, `DatasetService`	Data Workspace
mlcore	Model registry, training, background jobs, artifact storage	`ModelRegistry`, `ModelSpec`, `TrainingService`, `JobRunner`, `ArtifactStore`	Model Selection and Training
evaluation	Metrics, confusion matrix, feature importance	`MetricsCalculator`, `ConfusionMatrixBuilder`, `FeatureImportanceService`	Evaluation and Explainability
experiments	Run history, comparison, report export	`Run`, `Metric`, `ExperimentService`, `ReportExporter`	Experiment Management and Reporting
audit	Audit trail, structured logging, request IDs, health check	`AuditLog`, `audit.record()`, `JsonFormatter`, `RequestIdMiddleware`, `health_view`	Logging, Monitoring and Audit

Module	Responsibility	Key classes and functions	Component (3.4)
api	URL routing, controllers, request validators, error mapping	`UploadView`, `TrainView`, `EvaluateView`, `RunsView`, `AdminView`, `ApiError`, `error_handler`	API Gateway and Controllers; Input Validation
web	Templates, static JavaScript, Chart.js charts	Page templates, `api.js` (fetch wrapper with CSRF), `charts.js`, `poll.js`	Web UI

4.1.3 Pipeline design

The ML workflow is a pipe-and-filter pipeline. Upload and profiling run when a file arrives; the remaining stages run together inside one training job.

Stage	Input	Output	Failure handling
Validate	Uploaded file, user	Accepted file or `ValidationError`	Reject with 400, 413 or 415; nothing is stored.
Profile	Stored CSV	Rows, columns, types, missing counts, unique counts	Unparseable file: delete it and return 400.
Prepare	Dataset, target column, options	Cleaned feature matrix: missing values dropped, or filled with mean or median (numeric) or most frequent value (categorical); categories one-hot encoded	Unknown column or unsupported option returns 400; target with missing values is rejected.
Split	Prepared data, test size, seed, stratify flag	Train and test sets	Too few rows per class for a stratified split returns 422 with the reason.
Train	Train set, model name, parameters	Fitted model and artifact path	Exception or timeout marks the run FAILED; partial artifact is deleted.
Evaluate	Fitted model, train and test sets	Metrics, confusion matrix, feature importance where supported	Metric not defined for the data (for example ROC-AUC on a single class) is omitted and noted.
Record	Configuration and results	`Run` and `Metric` rows, status COMPLETED	Database error marks the run FAILED and is logged.

Preprocessing parameters (imputation values, encoders, scaler) are fitted on the training split only and then applied to the test split, which prevents data leakage (risk R4 in SAD Section 3.7).

4.1.4 Data model

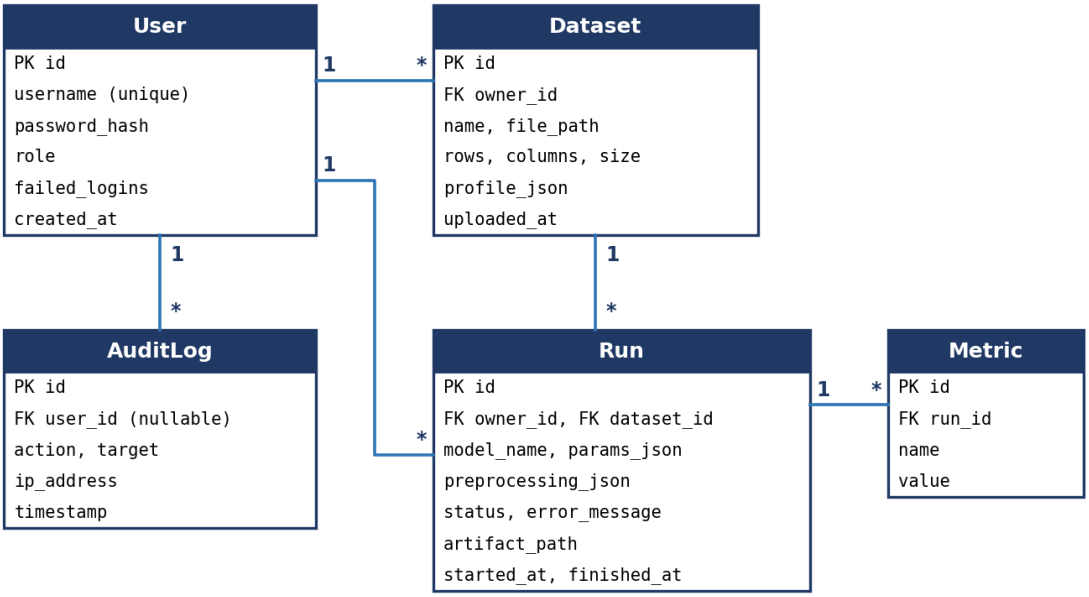


Figure 2: Entity-relationship diagram of the relational database

Run status follows a fixed life cycle: QUEUED, then RUNNING, then TRAINED, then COMPLETED. A run moves to FAILED from any state on error or timeout. Metric names carry the split they were computed on (for example `test_accuracy` and `train_accuracy`) so both are stored in one table.

Files are stored under a per-user directory outside the web root: `data/{user_id}/datasets/{uuid}.csv` and `data/{user_id}/models/{run_id}.joblib`. File names are generated by the server; the user's file name is kept only as display text.

4.1.5 Key design decisions

ID	Decision	Reason
DD1	Training runs in a background thread; status is stored in the database	Keeps the UI responsive for long jobs and survives page reloads; no extra infrastructure.
DD2	Single `/api/train` request carries model, parameters, preprocessing and split	One record reproduces one run; avoids leakage; fewer endpoints to test.
DD3	Artifacts are loaded only from paths registered by the system for the owner	Prevents unsafe deserialisation of user-supplied files (risk R3, MLE-SR-004).
DD4	Another user's resource returns 404, not 403	Does not reveal that the resource exists (MLE-SR-005).
DD5	Admins manage users and dataset records but cannot read dataset contents or train on other users' data (MLE-F-016)	Keeps confidentiality of user data (G2); required by the SRS.

ID	Decision	Reason
DD6	Classification metrics only in v1.0 (accuracy, precision, recall, F1, ROC-AUC)	Matches the SRS (MLE-F-010): macro-averaged for more than two classes; regression is out of scope.

4.2 Sequence Diagrams

Three sequence diagrams cover the flows that carry most of the requirements: login (security), dataset upload (data handling and validation) and training with metric viewing (the core ML flow). Solid arrows are calls, dashed arrows are returns, and red arrows are error paths.

4.2.1 Login and authentication (DS-Auth-01)

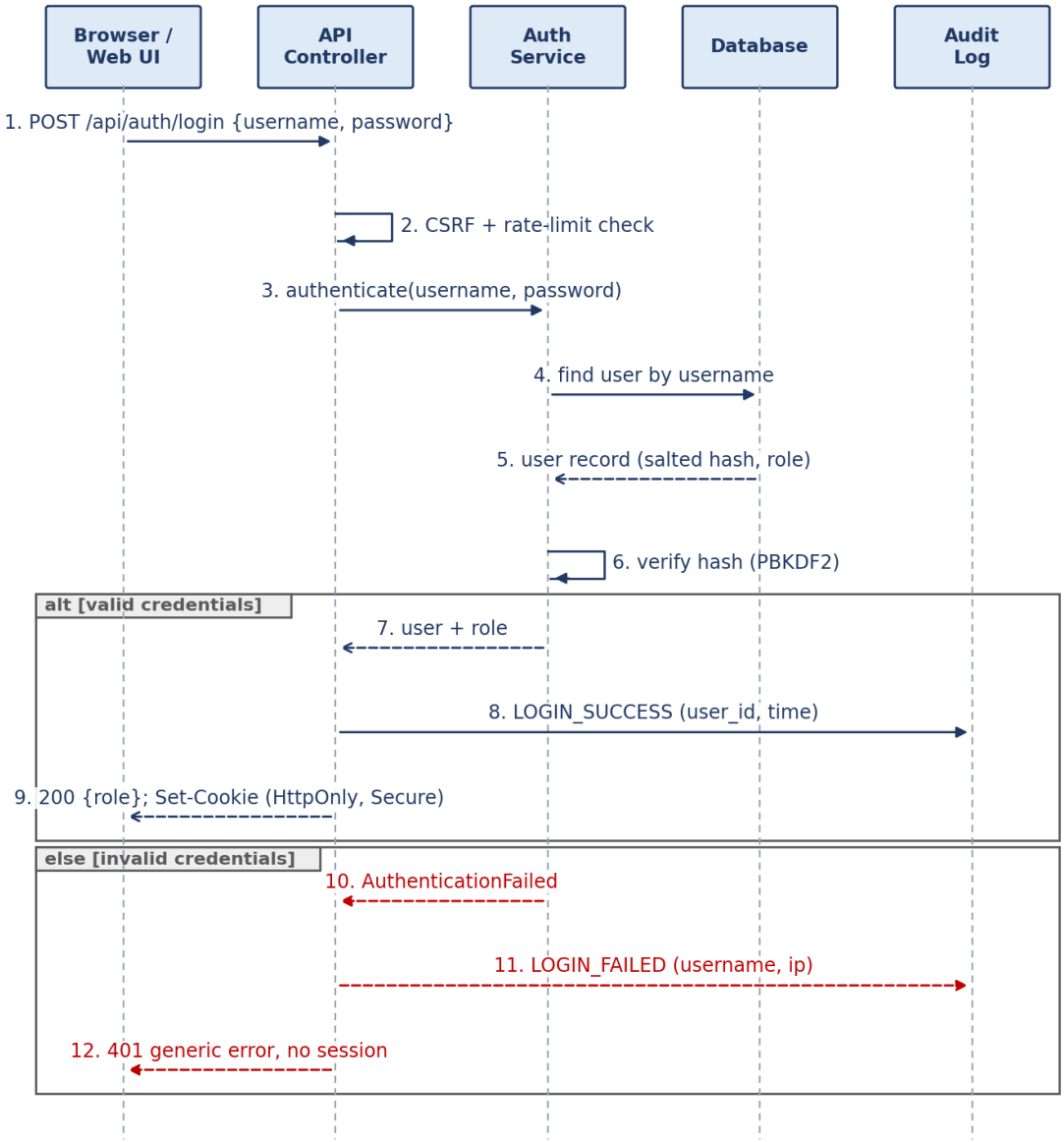


Figure 3: Sequence diagram, login and authentication

- The controller checks the CSRF token and the login rate limit before any credential check.
- `AuthService` loads the user and verifies the password against the stored PBKDF2 hash. Plain passwords are never stored or logged (MLE-SR-001).
- On success, the session cookie is set with HttpOnly and Secure flags and the response carries the role so the UI shows the right menu.
- On failure, the response is the same generic message whether the username or the password was wrong; repeated failures return 429. The attempt is written to the audit log.

4.2.2 Upload a dataset (DS-Data-01)

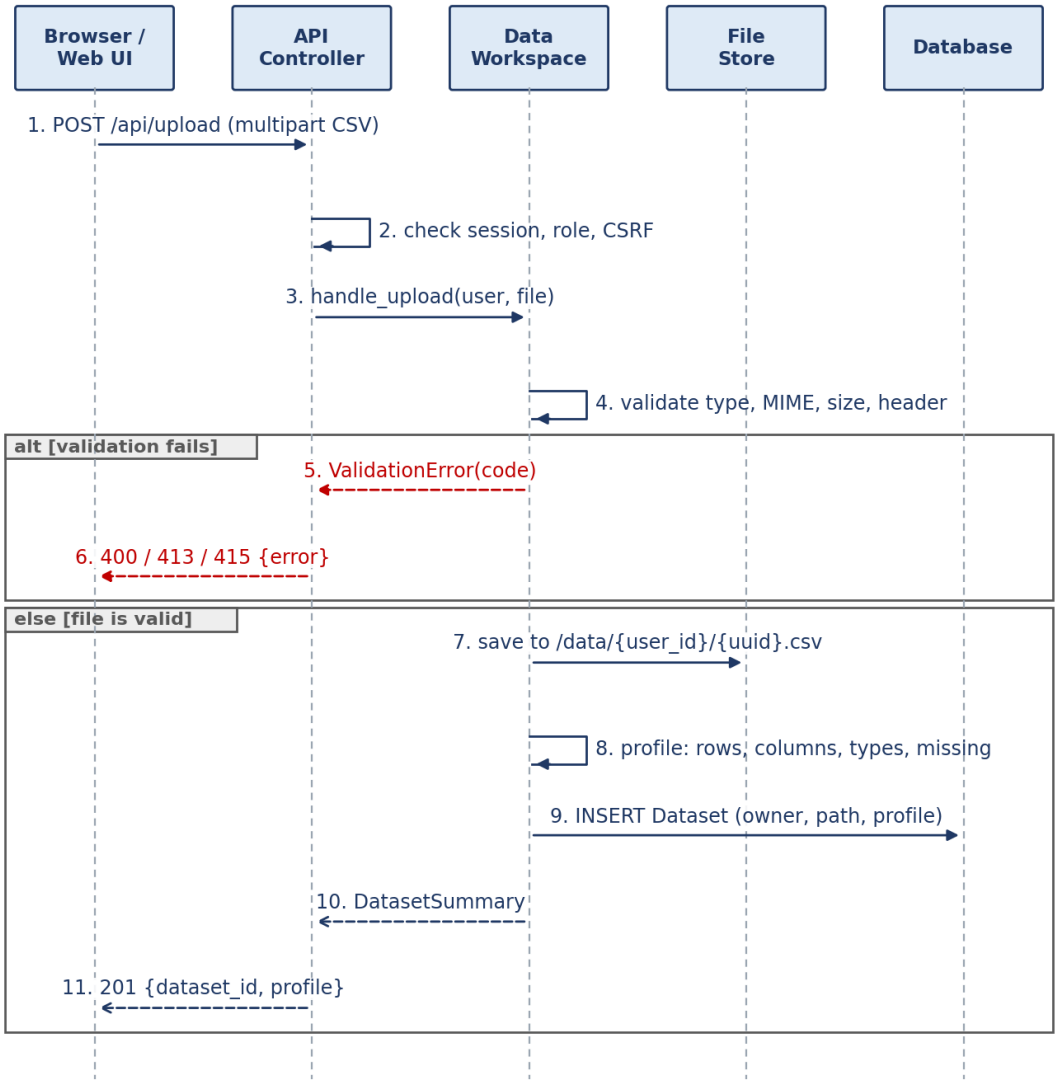


Figure 4: Sequence diagram, dataset upload and profiling

- Authentication, role and CSRF are checked before the file is read.
- `UploadValidator` checks extension, MIME type, size, encoding and header row before anything is saved (MLE-SR-003).

- The file is saved under a server-generated name in the owner's directory, profiled, and a `Dataset` row is created with the profile as JSON.
- Failure returns a specific code (400, 413 or 415) and stores nothing.

4.2.3 Train a model and view metrics (DS-Model-03, DS-Eval-01)

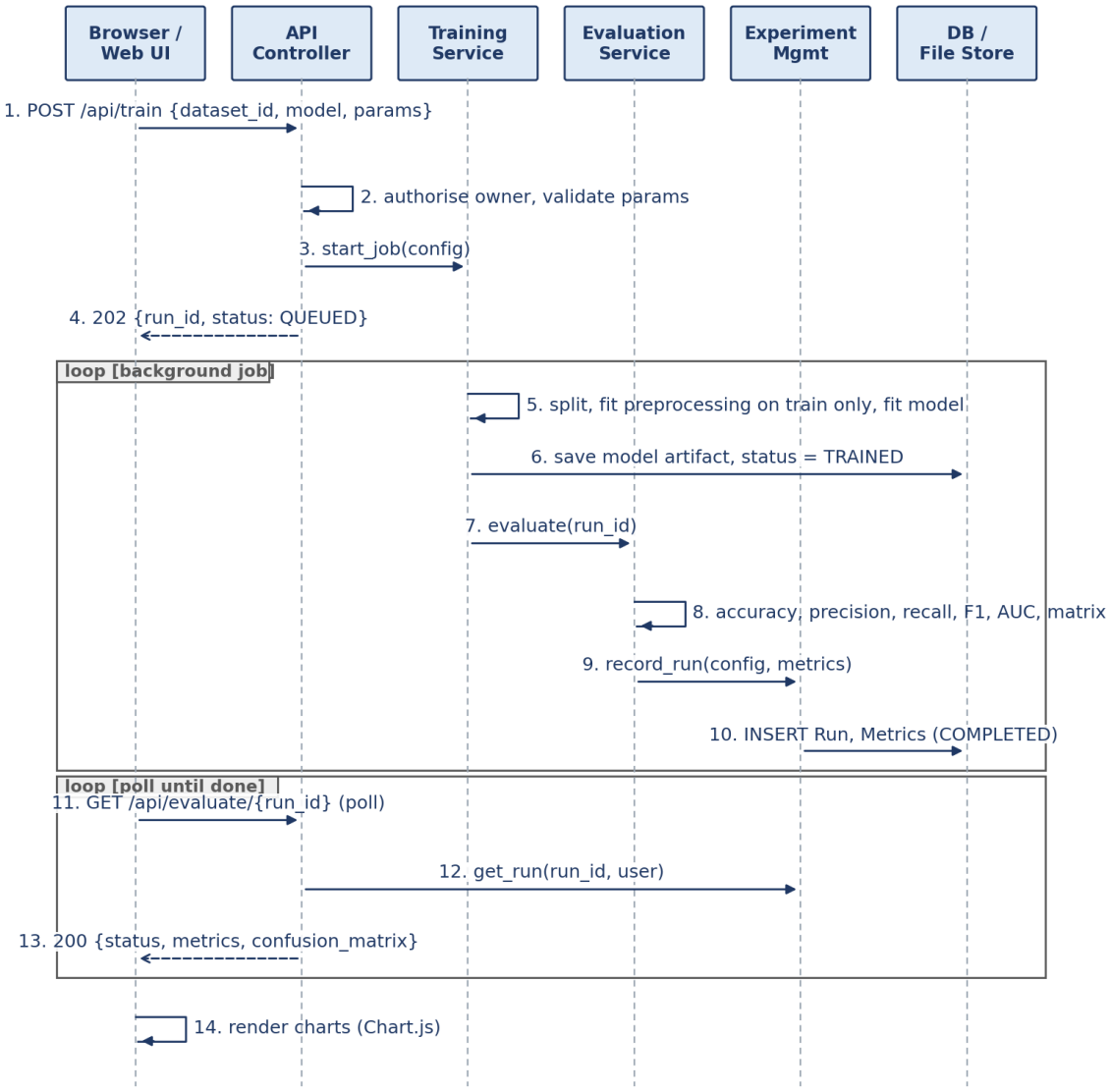


Figure 5: Sequence diagram, training, evaluation and viewing metrics

- `/api/train` returns 202 immediately with a run ID; the job runs in the background so the UI stays responsive (MLE-NF-002).
- The job splits the data, fits preprocessing on the training split only, trains the model and saves the artifact.
- Evaluation computes metrics on the test set (and train set for comparison) and `ExperimentService` records the run.

- The browser polls `/api/evaluate/{run_id}` until the status is COMPLETED or FAILED, then draws the charts.

4.3 API Design

4.3.1 Conventions

Topic	Convention
Base path and format	`/api/`, JSON in UTF-8. The upload endpoint uses `multipart/form-data`. Unversioned in v1.0.
Authentication	Django session cookie (HttpOnly, Secure, SameSite=Lax). Unauthenticated calls return 401.
CSRF	State-changing requests (POST, PATCH, DELETE) must send the token in the `X-CSRFToken` header.
Transport	HTTPS with TLS 1.2 or higher in deployment (MLE-SR-002), terminated at a reverse proxy in front of Django.
Identifiers and time	Integer IDs. Timestamps in ISO 8601 UTC, for example `2026-10-20T09:30:00Z`.
Pagination	List endpoints accept `page` (default 1) and `page_size` (default 20, maximum 100).
Authorisation	Deny by default. A resource that belongs to another user returns 404.
Limits	Request body over 10 MB returns 413. At most 120 requests per minute per user (per IP when anonymous); excess returns 429 (MLE-SR-006).
Errors	One envelope for every error (see below) and the HTTP status codes listed in Section 4.4.2.

Error envelope used by every endpoint:

```
{ "error": {
    "code": "VALIDATION_ERROR",
    "message": "Test size must be between 10% and 50%.",
    "details": { "field": "split.test_size" },
    "request_id": "9f3c2a1e" } }
```

4.3.2 Endpoint summary

Method	Path	Purpose	Roles	Requirement / DS-ID
POST	/api/auth/register	Create an account	Public	MLE-F-001 / DS-Auth-01
POST	/api/auth/login	Log in and start a session	Public	MLE-F-001, SR-001 / DS-Auth-01
POST	/api/auth/logout	End the session	Any logged-in user	MLE-F-001
POST	/api/upload	Upload a CSV dataset	DS, Student	MLE-F-003, SR-003 / DS-Data-01

Method	Path	Purpose	Roles	Requirement / DS-ID
GET	/api/datasets	List own datasets	DS, Student	MLE-F-004
GET	/api/datasets/{id}	Preview rows and profile	DS, Student	MLE-F-004 / DS-Data-02
DELETE	/api/datasets/{id}	Delete own dataset	DS, Student	MLE-SR-005
GET	/api/models	Model catalogue and parameter schema	DS, Student	MLE-F-007, F-008 / DS-Model-01, 02
POST	/api/train	Start a training run	DS, Student	MLE-F-005 to F-009 / DS-Data-03, 04, DS-Model-03
GET	/api/evaluate/{run_id}	Run status, metrics, confusion matrix	DS, Student	MLE-F-010 to F-012 / DS-Eval-01 to 03
GET	/api/runs	List own runs (filters)	DS, Student	MLE-F-013 / DS-Exp-01
GET	/api/runs/{id}	One run with configuration	DS, Student	MLE-F-013
GET	/api/runs/compare?ids=1,2	Side-by-side comparison	DS, Student	MLE-F-014 / DS-Exp-02
GET	/api/runs/{id}/report?format=csv	Export a results report	DS, Student	MLE-F-015 / DS-Exp-03
DELETE	/api/runs/{id}	Delete own run and artifact	DS, Student	MLE-SR-005
GET	/api/admin/users	List users and roles	Admin	MLE-F-016 / DS-Admin-01
PATCH	/api/admin/users/{id}	Change role; deactivate or reactivate an account	Admin	MLE-F-002, F-016
GET	/api/admin/datasets	List dataset records (metadata only)	Admin	MLE-F-016
DELETE	/api/admin/datasets/{id}	Remove a dataset record	Admin	MLE-F-016
GET	/api/admin/audit	View the audit log (no raw data)	Admin	MLE-F-016, SR-005
GET	/api/health	Health check (no data)	Public	Section 4.4.5

DS = Data Scientist. Data Scientist and Student have the same API access to their own data and differ only in the active-job limit (Student 1, Data Scientist 2; MLE-SR-006). Admin cannot upload, train or read dataset contents (SRS Table 4.1-1). Preview data in GET /api/datasets/{id} is limited to the first 20 rows.

4.3.3 POST /api/upload

Uploads one CSV file and returns its profile. Request is `multipart/form-data` with the fields `file` (required) and `name` (optional display name).

Validation rule (SRS Table 4.2-1)	Detail
Extension and type	`.csv` only; MIME type `text/csv` or `text/plain`; file content must parse as CSV.
Size	At most 10 MB (exactly 10 MB is accepted); empty files rejected (MLE-NF-001).

Validation rule (SRS Table 4.2-1)	Detail
Shape	Header row required; 20 to 50,000 data rows; 2 to 100 columns; every row has the same number of fields as the header.
Encoding	UTF-8, comma-delimited.
Quota	At most 20 stored datasets per user.
Target column (checked at training)	2 to 20 distinct values; every class has at least 2 rows.
Content safety	Cells beginning with =, +, - or @ are flagged and neutralised in any export; the file is parsed as data and never executed.

Success response (201):

```
{ "dataset_id": 7, "name": "titanic.csv", "size_bytes": 61194,
  "rows": 891, "columns": 12,
  "profile": [
    { "column": "age", "type": "numeric", "missing": 177, "unique": 88 },
    { "column": "sex", "type": "categorical", "missing": 0, "unique": 2 } ],
  "uploaded_at": "2026-10-20T09:30:00Z" }
```

Status	Code	When
201	-	File accepted and profiled.
400	VALIDATION_ERROR	Missing file, empty file, no header, too few rows or columns, too many rows or columns, not parseable as CSV.
401	AUTH_REQUIRED	No valid session.
413	PAYLOAD_TOO_LARGE	File larger than 10 MB.
415	UNSUPPORTED_MEDIA_TYPE	Wrong extension or MIME type.
422	UNPROCESSABLE	The user already has 20 stored datasets.
429	RATE_LIMITED	Too many uploads in a short time.

4.3.4 GET /api/models and POST /api/train

GET `/api/models` returns the catalogue so the UI builds the parameter form from data rather than hard-coded fields. The v1.0 catalogue is fixed by the SRS (Table 4.3-1):

Model (`model` value)	Class	Hyperparameters (default, range)
logistic_regression	LogisticRegression	C (1.0, 0.01 to 100); max_iter (200, 100 to 1000)
knn	KNeighborsClassifier	n_neighbors (5, 1 to 50); weights (uniform or distance)
decision_tree	DecisionTreeClassifier	max_depth (no limit, 1 to 50); min_samples_split (2, 2 to 20)
random_forest	RandomForestClassifier	n_estimators (100, 10 to 200); max_depth (no limit, 1 to 50)

Request to POST `/api/train` (JSON):

```
{ "dataset_id": 7,
  "target_column": "survived",
  "model": "random_forest",
  "params": { "n_estimators": 100, "max_depth": 8 },
  "preprocessing": { "missing": "median", "categorical": "one_hot" },
  "split": { "test_size": 0.2, "random_state": 42 } }
```

Response (202 Accepted):

```
{ "run_id": 12, "status": "QUEUED",
  "links": { "run": "/api/runs/12", "evaluate": "/api/evaluate/12" } }
```

Status	Code	When
202	-	Run created and queued.
400	VALIDATION_ERROR	Unknown model, parameter outside its allowed range, unknown column, invalid option, test share not between 10% and 50% (steps of 5%), target column that breaks the target rule.
401	AUTH_REQUIRED	No valid session.
404	NOT_FOUND	Dataset does not exist or belongs to another user.
422	UNPROCESSABLE	Request is valid but cannot be run, for example a stratified split with too few rows per class.
429	RATE_LIMITED	Active-job limit reached (Student 1, Data Scientist 2) or more than 120 requests per minute.

4.3.5 GET /api/evaluate/{run_id}

Returns the status of a run and, once finished, its results. While the run is QUEUED or RUNNING the `metrics` field is null and the call still returns 200, so the browser can poll without treating progress as an error.

```
{ "run_id": 12, "status": "COMPLETED", "dataset_id": 7, "model": "random_forest",
  "metrics": {
    "test":  { "accuracy": 0.91, "precision": 0.89, "recall": 0.90, "f1": 0.89,
"roc_auc": 0.95 },
    "train": { "accuracy": 0.97, "precision": 0.96, "recall": 0.97, "f1": 0.96,
"roc_auc": 0.99 } },
  "averaging": "macro",
  "confusion_matrix": { "labels": ["0", "1"], "matrix": [[85, 5], [8, 102]] },
  "feature_importance": [ { "feature": "age", "importance": 0.31 } ],
  "started_at": "2026-10-20T09:31:00Z", "finished_at": "2026-10-20T09:31:07Z",
  "error": null }
```

Metrics are macro-averaged when the target has more than two classes, and `roc_auc` is present only when the model provides probability estimates. If the run FAILED, `status` is FAILED, `metrics` is null

and `error` holds a safe `code` and `message`. A run that does not exist, or belongs to another user, returns 404.

4.3.6 Runs, comparison and export

- `GET /api/runs?dataset_id=7&model=random_forest&page=1` returns the user's runs newest first with model, key metrics and status.
- `GET /api/runs/compare?ids=12,13` accepts 2 to 4 run IDs of the same user on the same dataset and target column (a mixed selection returns 422 with the reason) and returns their configurations and metrics aligned for a side-by-side table.
- `GET /api/runs/{id}/report?format=csv` returns a downloadable CSV with configuration and metrics. PDF is not committed by the SRS (MLE-F-015) and is built only if time remains (risk R8). Cell values that start with =, +, - or @ are neutralised.

4.3.7 Authentication and admin endpoints

`POST /api/auth/login` takes `{ "username": "...", "password": "..." }` and returns 200 with `{ "user": { "id": 3, "username": "ds_test1", "role": "DATA_SCIENTIST" } }` and the session cookie. A wrong username or password returns 401 `INVALID_CREDENTIALS` with the same message in both cases, and a deactivated account cannot log in. After 5 failed logins within 10 minutes the account is blocked for 15 minutes (429 `ACCOUNT_LOCKED`). A session ends after 30 minutes of inactivity.

`POST /api/auth/register` takes `username` (3 to 30 letters, digits or underscores), `email`, `password` (at least 8 characters with a letter and a digit) and `role` (STUDENT or DATA_SCIENTIST). An Admin account is created only by an existing Admin or at installation. Passwords are stored as salted PBKDF2 hashes. Admin endpoints return 403 `FORBIDDEN` to non-admin users.

4.4 Error Handling, Logging and Monitoring

4.4.1 Principles

- **Validate at the boundary.** Controllers and `UploadValidator` reject bad input before it reaches services or storage.
- **Fail safe.** On error, no partial dataset, artifact or run record is left behind.
- **Tell the user what to do, tell the log what happened.** The user sees a short, plain message and a request ID; the log holds the technical detail. Stack traces and internal paths are never returned.
- **One handler.** A single `error_handler` maps `ApiError` subclasses and unexpected exceptions to the error envelope, so no controller formats errors itself.

4.4.2 Error catalogue

Code	HTTP	When	Message shown to the user
VALIDATION_ERROR	400	Input fails a validation rule	Names the field and the rule, for example: Test size must be between 10% and 50%.
AUTH_REQUIRED	401	No valid session	Please log in to continue.
INVALID_CREDENTIALS	401	Wrong username or password	Invalid username or password.

Code	HTTP	When	Message shown to the user
FORBIDDEN	403	Role not allowed	You do not have permission to do this.
NOT_FOUND	404	Missing resource or one owned by another user	We could not find that item.
PAYLOAD_TOO_LARGE	413	File over the size limit	The file is larger than 10 MB.
UNSUPPORTED_MEDIA_TYPE	415	Wrong file type	Only CSV files are supported.
UNPROCESSABLE	422	Valid request that cannot be run	States the reason, for example: too few rows in a class to split.
RATE_LIMITED	429	Too many requests or jobs	Too many requests. Please wait and try again.
ACCOUNT_LOCKED	429	5 failed logins within 10 minutes	Too many failed logins. Try again in 15 minutes.
TRAINING_FAILED	Run status	Training or evaluation raised an error	Training did not complete. Check your data and options, then try again.
TRAINING_TIMEOUT	Run status	Job exceeded the time limit	Training took too long. Try a smaller dataset or a simpler model.
INTERNAL_ERROR	500	Unexpected server error	Something went wrong. Quote request ID {id} if you contact support.

4.4.3 Failure handling for training jobs

- **Timeout.** Each job has a time limit (300 s, MLE-SR-006(d); the performance target in MLE-NF-002 is 60 s for 50,000 rows and 20 feature columns). On timeout the run is marked FAILED with `TRAINING_TIMEOUT`.
- **Exceptions.** Any exception sets the run to FAILED with a sanitised message; the full exception goes to the log with the request ID.
- **Cleanup.** A partial artifact is deleted when a run fails.
- **Restart recovery.** At start-up, runs still marked QUEUED or RUNNING are set to FAILED with the message "interrupted by server restart".
- **Quota.** Each user may have at most 2 queued or running jobs (Student: 1), which limits denial-of-service from one account (MLE-SR-006(c)). A failed job never affects other users (MLE-NF-006).

4.4.4 Logging and audit

The application writes one JSON object per line to standard output (collected by Docker) and to a rotating file. Security-relevant events are also stored in the `AuditLog` table so the Admin can review them.

Field	Content
Common fields	`timestamp`, `level`, `request_id`, `user_id`, `method`, `path`, `status`, `duration_ms`, `event`.
Levels	INFO for normal events, WARNING for rejected input and denied access, ERROR for failed jobs and unexpected exceptions.
Never logged	Passwords, session IDs, CSRF tokens, dataset cell values, file contents or model parameters that identify private data.
Retention (proposed)	Daily rotation, 14 days kept for files; audit rows kept for the project period.

Audit events (MLE-SR-005): `REGISTER`, `LOGIN_SUCCESS`, `LOGIN_FAILED`, `LOGOUT`, `UPLOAD`, `DATASET_DELETE`, `TRAIN_START`, `TRAIN_DONE`, `TRAIN_FAIL`, `REPORT_EXPORT`, `ACCESS_DENIED`, `ADMIN_USER_UPDATE`, `ADMIN_DATASET_DELETE`. Each stores the user ID, target, IP address and timestamp.

4.4.5 Monitoring

- **Health check.** `GET /api/health` returns 200 when the database answers and the file store is writable, and 503 otherwise. It returns no user data. Docker uses it as the container health check.
- **Operational indicators.** Request count and error rate, training duration, number of queued jobs and number of failed runs, taken from the structured logs and shown on the Admin panel as a recent-activity summary.
- **Pipeline checks.** GitHub Actions runs tests, linting and coverage on every pull request; static analysis (SonarQube or flake8) reports code quality, matching the security validation required by the guidelines.
- **Alert rules for the demo.** A failed health check, 5 failed logins in 10 minutes (account blocked for 15 minutes), or a failed-run rate above 20% are shown as warnings on the Admin panel.

4.5 UX Design

4.5.1 Principles

- **Guided workflow.** The main task is a clear sequence: upload, prepare, choose a model, train, read results. Each screen shows the next step.
- **Visible status.** Uploads and training show progress; a run's state is always readable (queued, running, completed or failed).
- **Safe defaults.** The model form is pre-filled with the default parameters, and the preparation step defaults to median imputation, one-hot encoding and an 80/20 split.
- **Plain-language errors.** Messages follow Section 4.4.2 and say what to do next.
- **Consistency and accessibility.** One layout, one navigation bar, labelled fields, keyboard operation, sufficient colour contrast, and text or table alternatives for charts.

4.5.2 Screen map

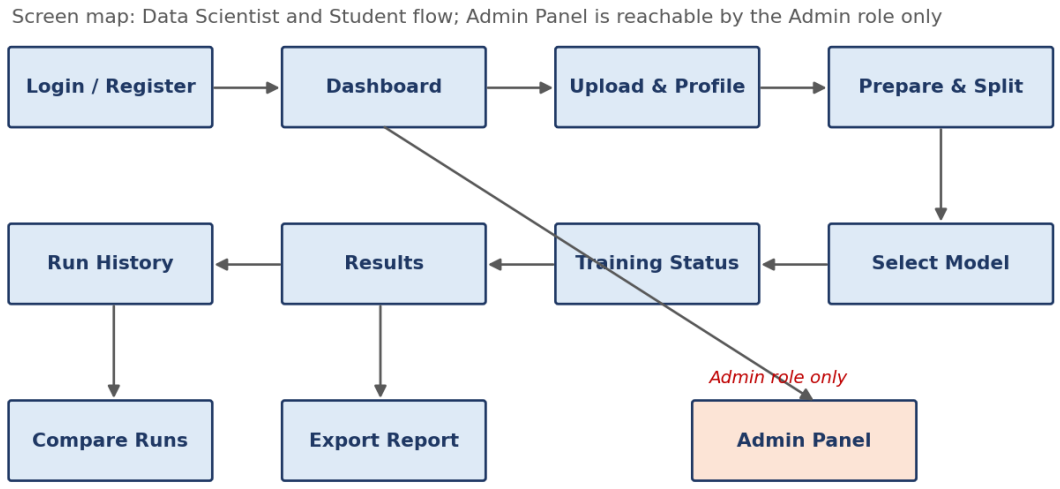


Figure 6: Screen map and navigation flow

4.5.3 Screen inventory

Screen	Purpose	Main elements	Requirement
Login / Register	Sign in or create an account	Username, password, role choice on registration, error banner	MLE-F-001
Dashboard	Starting point	Recent datasets and runs, shortcut to upload, Admin link for Admin only	MLE-F-002
Upload and Profile	Add and inspect a dataset	File drop area, size hint, upload progress, preview of first rows, profile table	MLE-F-003, F-004
Prepare and Split	Set preprocessing and split	Target column, missing-value method, one-hot encoding, test share 10% to 50% (default 20%), random seed (default 42)	MLE-F-005, F-006
Select Model	Choose and configure a model	Catalogue cards, parameter form built from the schema, Train button	MLE-F-007, F-008
Training Status	Show progress	Status badge, progress indicator, polling every 2 s, error message if failed	MLE-F-009
Results	Read evaluation	Metric tiles, confusion matrix, metric bars, feature importance, Compare and Export buttons	MLE-F-010 to F-012
Run History	Browse past runs	Filterable table, select two or more runs, delete	MLE-F-013
Compare Runs	Side-by-side view	Table of configurations and metrics with best value highlighted	MLE-F-014
Admin Panel	Manage the platform	User list with role change and disable, dataset records, audit log, health	MLE-F-016

4.5.4 Wireframes

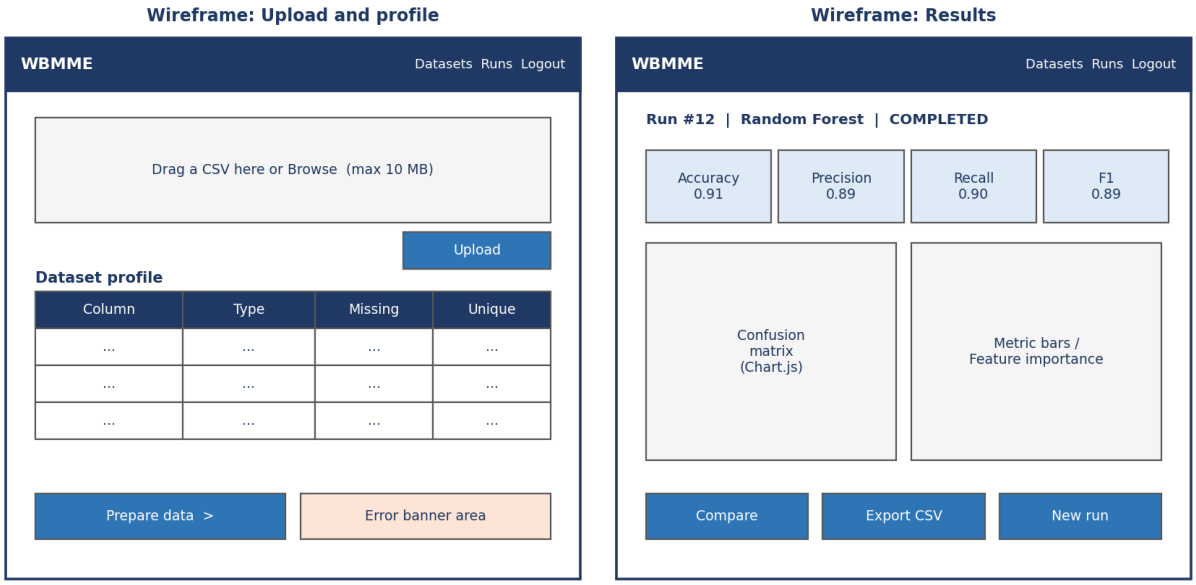


Figure 7: Wireframes for the Upload and Profile screen and the Results screen (sample values)

4.5.5 Feedback and usability

- Field errors appear next to the field; request-level errors appear in a banner at the top of the screen; success appears as a short confirmation.
- Delete actions for datasets, runs and users ask for confirmation.
- Empty states explain what to do (for example, an empty Run History links to Upload).
- Usability target for MLE-NF-005 (proposed measure for the Test Plan): a first-time user completes login, upload, training and viewing metrics within 10 minutes without help.
- Layouts are designed for desktop browsers only (SRS 2.4); mobile is out of scope.

4.6 Open Issues & Next Steps

Questions about IDs, limits, models, roles and export were settled by SRS Part B (Member 2). The items below remain open.

ID	Issue	Impact	Proposed resolution	Decide by
OI-1	Test Plan (Member 1) has its own copy of the 10 test cases	Test Plan 13, Member 4 test cases file	Resolved on merge: Member 4's ten test cases are Appendix A of the Test Plan, and its Section 13 carries the design references.	Closed 13-10-2026
OI-2	Background jobs: threads or Celery	4.4.3, deployment	Threads for v1.0; Celery behind `JobRunner` if time allows	Sprint 1 planning

ID	Issue	Impact	Proposed resolution	Decide by
OI-3	Rate limiting (120 requests per minute per user and per IP)	4.3.1, MLE-SR-006	Django middleware or a rate-limit library; agree in Sprint 1	Sprint 1 planning
OI-4	Module ownership per team member (guidelines require one full functionality each)	4.1.2, GitHub issues	Assign modules to members at Sprint 1 planning	Sprint 1 planning
OI-5	HTTPS applies to the deployed build only (MLE-SR-002); local build is exempt	Security tests SV-02	TLS at a reverse proxy in the Docker setup	Sprint 2
OI-6	SHAP explanations and PDF export are not committed by the SRS	MLE-F-012, F-015	Build only if time remains after Sprint 2	Sprint 2

5. Appendices

5.1 Glossary

WBMME, SRS, SAD, STP, RTM, API, CSV, ML, MVT, RBAC, CSRF, STRIDE, TLS, PBKDF2, DS-ID, Run, Artifact and Pipeline are defined in Sections 1.3 and 3 of this document and in SRS Section 1.4.

5.2 References

IEEE 1016-2009, IEEE 42010, OWASP Top 10 and ASVS, NIST SP 800-160, Django 5 and scikit-learn documentation.

5.3 Tools

Matplotlib (diagrams), draw.io and PlantUML (alternatives), Postman and Swagger (API), GitHub Actions, pytest, Docker.

Appendix A: Design reference index and traceability

This table extends the RTM with the design location and is the single place to edit when requirement IDs change. Test case IDs are from the Test Plan (Section 13). "Planned" means the case is scheduled for Sprint 1 or 2.

Req ID	DS-ID	Design location	Test case(s)
MLE-F-001	DS-Auth-01	4.2.1 login sequence; 4.3.7 auth endpoints; module `accounts`	TC-AUTH-01
MLE-F-002	DS-Auth-02	4.3.2 roles; `role_required`; 4.3.7	TC-SEC-01
MLE-F-003	DS-Data-01	4.2.2 upload sequence; 4.3.3 `/api/upload`	TC-DATA-01, TC-DATA-02
MLE-F-004	DS-Data-02	4.1.3 Profile stage; `GET /api/datasets/{id}`	TC-DATA-01
MLE-F-005	DS-Data-03	4.1.3 Prepare stage; `preprocessing` in 4.3.4	Planned (Sprint 1)
MLE-F-006	DS-Data-04	4.1.3 Split stage; `split` in 4.3.4	TC-MODEL-01
MLE-F-007	DS-Model-01	4.3.4 `GET /api/models`; `ModelRegistry`	TC-MODEL-01
MLE-F-008	DS-Model-02	4.3.4 `params`; registry parameter ranges	TC-MODEL-01
MLE-F-009	DS-Model-03	4.2.3 train sequence; 4.3.4 `/api/train`; 4.4.3	TC-MODEL-01
MLE-F-010	DS-Eval-01	4.3.5 `/api/evaluate`; `MetricsCalculator`	TC-EVAL-01
MLE-F-011	DS-Eval-02	4.3.5 confusion matrix; 4.5.3 Results screen	TC-EVAL-01
MLE-F-012	DS-Eval-03	4.3.5 `feature_importance`	Planned (Sprint 2)
MLE-F-013	DS-Exp-01	4.1.4 `Run`, `Metric`; 4.3.6 `/api/runs`	TC-EXP-01
MLE-F-014	DS-Exp-02	4.3.6 `/api/runs/compare`; 4.5.3 Compare screen	TC-EXP-01
MLE-F-015	DS-Exp-03	4.3.6 CSV report export	TC-DATA-02 (step 3); full test Sprint 2
MLE-F-016	DS-Admin-01	4.3.2 and 4.3.7 admin endpoints; Admin Panel screen	TC-SEC-01

Req ID	DS-ID	Design location	Test case(s)
MLE-NF-001	-	4.3.3 size and row limits; streamed validation	TC-PERF-01
MLE-NF-002	-	4.2.3 background job; 4.4.3 timeout and limits	TC-PERF-02
MLE-NF-003	-	4.3.1 pagination and stateless API; 4.5 light pages	TC-PERF-03
MLE-NF-004	-	Appendix B Dockerfile and compose file	Planned (Sprint 2)
MLE-NF-005	-	4.5 UX design; 4.5.5 usability target	Planned (Sprint 2)
MLE-SR-001	-	4.2.1 PBKDF2 hashes; 4.3.7	TC-AUTH-01, SV-01
MLE-SR-002	-	4.3.1 transport; OI-5	SV-02 (TLS scan)
MLE-SR-003	-	4.2.2 and 4.3.3 validation rules	TC-DATA-02, SV-03
MLE-SR-004	-	4.1.5 DD3; `ArtifactStore`	SV-04; code inspection
MLE-SR-005	-	4.1.4 owner rule; DD4; 4.4.4 audit events	TC-SEC-01, SV-05
MLE-SR-006	-	4.3.1 limits; 4.4.3 quota and 300 s timeout; 4.3.3 413	TC-MODEL-01 (job limit); SV-06 (Sprint 2)
MLE-SR-007	-	4.3.7 session, lockout; 4.3.1 CSRF header	TC-AUTH-01 (cookie flag), SV-07, SV-08
MLE-NF-006	-	4.4.1 and 4.4.3 failure handling and restart recovery	Fault-injection and restart test (Sprint 2)
MLE-NF-007	-	4.3.1 pagination (20 per page); 4.1.4 data model	Load test with seeded data (Sprint 2)
MLE-SO-001 to 003	-	Security objectives: 3.9; met by MLE-SR-001 to SR-007	Test Plan 5.1

Appendix B: Proposed repository layout

Layout of repository SE_498_515_534_911 that supports the module structure in 4.1.2 and lets members work on separate folders:

```
SE_498_515_534_911/
  manage.py, requirements.txt, Dockerfile, docker-compose.yml
  .github/workflows/ci.yml        # tests, lint, coverage on every pull request
  wbmme/                          # settings, urls, wsgi
  apps/
    accounts/  datasets/  mlcore/  evaluation/
    experiments/  audit/  api/
  templates/  static/ (js/, css/, vendor/chart.js)
  tests/ (unit/, integration/, system/)
  docs/ (SRS, SAD, Test Plan, RTM)
  data/                           # outside web root; created at runtime, not committed
```

Docker: one command, `docker compose up --build`, starts the system on Windows, Linux and macOS; pages are checked in current Google Chrome and Mozilla Firefox (MLE-NF-004).